INDIAN PEDIATRICS
AI ASSISTANT

STEP 4 | RETRIEVAL-AUGMENTED GENERATION

From cleaned pages to cited literature answers

This chapter explains how the POC turns traceable page text into searchable evidence, retrieves relevant passages, and constrains any generated answer to those passages.

Teaching manual installment 1.0 | October 2026

Project: Reusable medical-journal literature assistant
POC corpus: Indian Pediatrics, January 2025, Volume 62, Issue 1
Audience: AI students and developers learning to plan and build a grounded literature assistant

How to use this installment
These are the first chapters of a cumulative teaching manual. They explain decisions made so far and provide a reproducible way to understand them. The full 300+ page reference is intended to grow as later roadmap phases are completed.

Chapter map

- 18. Chunking - create manageable, page-linked passages.
- 19. Embeddings - represent meaning, with external processing gated.
- 20. Vector database - store searchable text and optional vectors.
- 21. Retrieval - apply metadata filters and evidence limits.
- 22. Hybrid search - combine lexical and semantic rankings.
- 23. RAG generation - draft only from retrieved passages.
- 24. Citations - display source metadata and validate labels.
- 25. No-answer logic - abstain when evidence or safe support is absent.

Learning objectives

- Explain how each chunk retains article, issue, source file, checksum, and page provenance.
- Rebuild the local lexical index without sending corpus text to a provider.
- Describe the difference between lexical matching and semantic vector similarity.
- Trace how retrieved evidence is labelled and exposed as a source citation.
- Recognize the current checkpoint boundary: local search is available; provider-backed semantic search and answer generation are not enabled.

18. Chunking

A chunk is a bounded passage used as the unit for retrieval and evidence. This POC chunks each cleaned source page independently, targeting 1,400 characters with an overlap of 180 characters. Keeping page boundaries makes provenance and citation review simple; long pages become multiple overlapping chunks.

18.1 What, why, and how

- What: split the Step 3 page records into small text passages, attach article metadata to each, and record any excluded flagged page.
- Why: an entire issue or article is too broad for focused retrieval, while a sentence-only fragment can lose context. A page-linked passage is a transparent starting compromise.
- How: `scripts/prepare_step4_chunks.py` reads `processing_manifest.csv`, article metadata, and ordered Step 3 JSONL page records. It writes `chunks.jsonl` and `chunk_exclusions.jsonl` under `data/processed/chunks/`.

18.2 Stable identifiers and provenance

- Each chunk ID is based on article ID, printed page (or PDF page fallback), and sequence on that page.
- Every chunk keeps article title, authors, section/category, content role, year, month, volume, issue, issue ID, printed page, physical page, source filename, source SHA-256, publisher URL, and page role.
- The systematic review flow diagram page previously flagged for visual review is excluded from chunks. The exclusion is logged, not erased; its source PDF and Step 3 review record remain available.
- The text passage itself is derived data. The original PDF remains the source of truth.

18.3 Chunking quality and trade-offs

Choice	Benefit	Limit
Page-scoped chunks	Page provenance remains exact and easy to cite.	A concept spanning a page break may be split.
1,400 character target	Keeps evidence focused while leaving paragraph context.	Not calibrated yet on a golden retrieval set.
180 character overlap	Preserves some context across splits within a page.	Can create near-duplicate results.
Long lines split on word boundaries	Avoids very large chunks and broken words.	Complex tables and columns need visual checking.

18.4 Common mistakes to avoid

- Do not remove page and source metadata after cleaning.
- Do not chunk flagged figure pages as if all labels were extracted.
- Do not choose chunk length based only on a popular tutorial; evaluate it against the journal's article layouts and Step 6 questions.
- Do not call character overlap a guarantee that a complete scientific argument is preserved.

19. Embeddings

An embedding is a numeric representation that can make passages with related meaning discoverable even when they use different words. The POC defines `text-embedding-3-small` as its intended embedding model, but it does not fabricate embeddings or use a lexical score as a semantic substitute.

19.1 What, why, and how

- What: send a query and candidate text passages to a configured embedding model and receive vectors of equal dimensions.
- Why: lexical search is good for exact names, phrases, and terms; semantic retrieval can help with paraphrases and related concepts.
- How: the optional `--with-embeddings` index build calls the provider in configured batches. It requires a local API key and the explicit `JOURNAL_ASSISTANT_ALLOW_EXTERNAL_TEXT_PROCESSING=true` gate.

Privacy and corpus permission gate
The default is disabled. An embedding request transmits article passages to an external service. A key alone does not grant permission. The project owner must review publisher/editor terms and approve this data flow before enabling it. The current run made no such API request.

19.2 Reproducibility and model identity

- Record the embedding provider, exact model, dimensions, index build time, and input checksum in a production manifest.
- Do not mix vectors from different models or dimensions in one index.
- If the model changes, re-embed the complete approved corpus and compare retrieval using a fixed evaluation set.
- Keep query embedding behavior consistent with document embedding behavior.

20. Vector database

The POC starts with SQLite: FTS5 stores searchable text, and an optional table stores float vectors and model dimensions. This is deliberately a small, local and inspectable index for one issue, rather than a commitment to a production-scale vector service.

20.1 What, why, and how

- What: persist one row per chunk and a parallel FTS record; optionally store the vector as a binary float array.
- Why: the first corpus is small enough for simple deployment and straightforward inspection. One SQLite file is easier to back up and demonstrate than a separately operated service.
- How: `SQLiteHybridStore` creates `data/indexes/journal_assistant.sqlite3`, enforces one row per chunk ID, and exposes lexical and cosine-similarity searches.

20.2 Why SQLite now; what may change later

Option	POC use	Later consideration
SQLite FTS5 + vector table	No separate server; local, portable, transparent.	Benchmark latency and vector count before scaling.
Dedicated vector database	Not needed for this one-issue POC.	Consider only if multi-tenant, large-scale corpus, access control or operations justify it.
In-memory mock vectors	Rejected because they would imply semantic capability without real embeddings.	Use deterministic fixtures only in isolated unit-level tests, never as production evidence.

20.3 Index integrity

- Rebuild is explicit and resets the derived index before loading the selected chunk file.
- A local lexical build stores zero vectors; the checkpoint calls this out rather than hiding it.
- Index rows keep full metadata and the chunk text so that search results can be audited and citations constructed without guesswork.
- Back up the raw corpus and generated manifests separately from the disposable derived SQLite index.

21. Retrieval

Retrieval selects passages to support a question before answer generation. It applies query text, allowed content roles, and optional metadata constraints such as author, year, volume, issue and article ID.

21.1 What, why, and how

- What: search chunk records, apply filters, rank matches and retain a small evidence set.
- Why: restricting evidence to the right article, time range, or content type helps relevance and reduces unrelated context.
- How: `retrieval.search` accepts a lexical query and optionally an already-computed vector, applies allowed roles/filters through the store, then returns up to the configured top six passages.

21.2 Recency and ranking

The metadata supports time filtering and future recency-aware ordering. This POC does not make newer research automatically more correct: evidence rank and publication date are distinct concepts. The current retrieval can constrain year ranges and exact issue/year/volume; a validated temporal boost should be added only after evaluation. The published year, month and issue remain visible in source cards.

21.3 Content roles

- Clinical evidence and clinical context are eligible for the default literature query; secondary sources receive a ranking penalty.
- Administrative material and advertisements are not included in the default evidence roles.
- Role filtering is not a substitute for judging study design, population, sample size, limitations, or subsequent research.
- The UI should distinguish a journal literature summary from clinical guidance.

22. Hybrid search

Hybrid search can combine lexical and semantic ranking. The POC uses reciprocal rank fusion (RRF), which combines positions rather than mixing raw scores with incomparable scales.

22.1 What, why, and how

- What: obtain lexical results from FTS5 and, when real query and document vectors exist, obtain semantic results from cosine similarity.
- Why: exact medical terms and author names benefit from lexical search; paraphrased topics may benefit from embeddings.
- How: RRF adds a reciprocal rank contribution from each ranked list, caps duplicate chunks per article, and applies a configurable lower multiplier to secondary-source results.

22.2 Current behavior

The current POC index contains only its local lexical list. Calling hybrid retrieval without a query vector therefore returns lexical matches only. This is an honest fallback, not semantic retrieval. After explicitly approved embedding generation, supply a query embedding to the same retrieval interface to combine both lists.

22.3 What to evaluate

- Recall at k: did relevant articles appear in the first k results?
- Precision at k: how many returned passages are actually relevant?
- Author/issue filter correctness: are out-of-filter rows excluded?
- Source diversity: does one long article crowd out all others?
- Compare lexical-only, semantic-only and hybrid on a reviewed question set before tuning weights or top-k.

23. RAG generation

Retrieval-augmented generation (RAG) supplies selected source passages to a language model for a grounded response. The provider adapter describes this boundary, but the present environment has no configured key and external text processing remains off.

23.1 What, why, and how

- What: combine a user question and retrieved evidence into a constrained request to the answer model.
- Why: the model is asked to synthesize evidence actually selected from this journal corpus rather than answer freely from its general training.
- How: the adapter labels each passage and instructs the model to stay within the evidence, describe publications as evidence rather than recommendations, and disclose insufficient or conflicting evidence.

23.2 Safety boundary

- The assistant is a literature-retrieval aid, not a clinical decision-maker.

- The initial patient-specific phrase detector is a narrow prototype boundary, not a complete medical safety classifier.
- Never rely on a prompt alone: keep source evidence visible, validate citations, and evaluate adversarial and ambiguous questions.
- Do not describe the POC as clinically validated or suitable for diagnosis or treatment.

24. Citations and source cards

Citations are built from the retrieved chunk's stored metadata rather than free-form model claims. Results receive labels such as [S1] and carry the article, authors, year, category, page, and publisher issue URL.

24.1 What, why, and how

- What: assign labels after ranking and include the same labels beside passages supplied for synthesis.
- Why: the reader needs to verify a statement against the source PDF; source pages and article identity let them retrace it.
- How: the answer validator accepts only citation labels present in the retrieved evidence list and rejects missing or invented labels.

24.2 Citation validation limits

A label check verifies that the model used a source ID that exists. It does not prove that the cited passage logically supports the sentence. Human review and claim-level evaluation remain necessary. A final desktop interface should let users open the source document at the cited page and should identify when only a physical PDF page is known.

25. No-answer logic and safe fallback

A reliable assistant sometimes declines to answer. No relevant search results, missing provider setup, or invalid citation output must not be disguised as a confident response.

25.1 What, why, and how

- No evidence: return an explicit message that the indexed journal corpus did not yield support.
- Provider unavailable: return the retrieved source passages and say generation is not configured.
- Patient-specific request detected: return the fixed literature-assistant boundary message.
- Invalid or absent citations: refuse the generated text and ask the user to inspect the actual retrieved sources.
- Conflicting or limited literature: the generation instructions ask the model to state that limitation rather than force a single conclusion.

25.2 Calibrating abstention

`minimum_evidence_count` currently expresses a starting configuration, not a clinically validated threshold. Step 6 must establish relevance and abstention thresholds on a reviewed question set. Do not interpret a lexical match as proof that a question is answered, or a zero-result response as proof that the journal contains no such information.

Reusable multi-journal configuration

The implementation separates corpus-specific data from shared retrieval logic. A new journal must receive its own explicit configuration, metadata mapping and permission review; it must never silently inherit Indian Pediatrics assumptions.

Portability checklist

- Create a journal profile for display name, canonical source URL, section taxonomy, identifier rules, volume/issue conventions, and allowed content roles.

- Map the new journal's metadata into the common schema while preserving original field names and values for audit.
- Use journal-prefixed article IDs and retain source checksums and access terms.
- Review file acquisition and external provider permission independently for the new publisher.
- Test extraction, chunking, filters, citations, role behavior and safety using that journal's own sample issue.
- Keep shared code generic; place per-journal behavior in named config and adapters rather than hard-coded conditionals.

Local reproduction and troubleshooting

All corpus processing and local lexical retrieval can be demonstrated without an AI API key. These commands operate from the Project folder.

Activate the virtual environment, then run the commands in order:

```
python scripts\prepare_step4_chunks.py
python scripts\build_step4_index.py
python scripts\check_step4.py
python scripts\ask_step4.py "ketorolac morphine sickle cell"
```

Symptom	Likely cause	Action
No chunks found	Step 3 derived records or metadata are missing.	Run Step 3 checkpoint; do not invent text to fill gaps.
Issue page is not in the result	Page may have a visual-review flag or wording differs.	Check exclusion ledger and original PDF; refine query or review the page.
Semantic results absent	No approved embeddings have been built.	Use local lexical retrieval or enable external processing only after policy review and configuration.
Answer generation unavailable	Missing key, missing SDK or external-processing gate remains off.	Keep source results available; configure provider only after approval.
Generated citation rejected	Output label does not match retrieved evidence.	Do not display as supported; inspect prompt and provider output, then re-evaluate.
A retrieved passage is irrelevant	Lexical overlap does not ensure relevance.	Record the query in the evaluation set; tune and measure rather than silently raising confidence.

Checkpoint results and handoff

The checker writes a repeatable report and distinguishes local retrieval checks from provider readiness. The current checkpoint must be read alongside the Step 3 documented figure-page exception.

Completion checklist

- Chunk files exist and have stable IDs, nonempty text and source provenance.
- Flagged figure pages are excluded and listed in the exclusion log.
- Local FTS index row count matches chunk count; a known corpus query returns source-linked results.
- A nonsense query yields no evidence; no-answer behavior is exercised without an API call.
- Author/year/issue metadata filters and allowed content roles are checked.
- No vectors or generated answers are claimed unless a real approved provider call created them.
- The report states provider/key/approval status without exposing credentials.

Current status and Step 5 handoff

Implementation foundation complete; full Step 4 is gated
Local page-aware chunking and lexical search can be checked. The provider-backed semantic index and answer-generation path have not been activated. The check report must show this as a hold before Step 5's quality/evaluation phase can treat the RAG path as complete.

This chapter describes implemented interfaces and a limited current POC run. It does not establish that answers are clinically safe, that all retrieved text is relevant, or that the publisher permits external AI processing. Preserve the source PDFs, processing reports, chunk exclusions and index build provenance for the next phase.

References and glossary

- Ver 1 - Complete Implementation Guide for POC.pdf, Phase 4 Retrieval-Augmented Generation and its activities 18-25.
- Ver 1 - Master Guide for POC.pdf, source traceability, grounded literature assistance, safety and reusable-journal principles.
- Project configuration: `config/rag.toml`; implementation: `src/journal_assistant/rag/`; local commands: `scripts/prepare_step4_chunks.py`, `scripts/build_step4_index.py`, `scripts/ask_step4.py`, and `scripts/check_step4.py`.

Glossary

Term	Plain-language meaning
Chunk	A bounded passage indexed for retrieval.
Embedding	A numeric vector intended to represent text meaning.
FTS5	SQLite's full-text search extension used for local lexical search.
Hybrid retrieval	Combining ranked results from lexical and semantic search.
RRF	Reciprocal rank fusion, a method for combining ranked lists.
Provenance	Information linking derived text to article, file, checksum and page.
Abstention	A deliberate no-answer when available evidence is insufficient or invalid.
Grounded answer	A response constrained to retrieved passages with traceable citations.